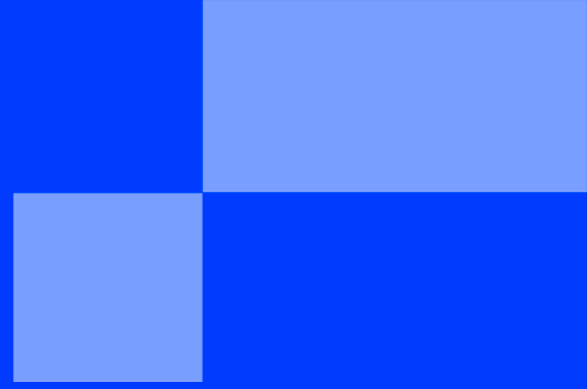




SUMMER SCHOOL

< PART 4 BOOTSTRAP - THE BACK-END
ENGINE />

SUMMER SCHOOL

< WEB DEVELOPMENT: BEHIND THE SCENES />

"It's not a bug. It's an undocumented feature."
— **Every Developer**

Introduction

In the previous modules, you built a beautiful, interactive Front-End using React. However, everything happened locally in the user's browser. If you refresh the page, the memory is wiped clean.

To build a real application, you need a **Back-End**.

A Back-End is a server running continuously somewhere in the world. Its job is to receive requests from clients (like your React app), process logic, interact with a database, and send responses back.

In this bootstrap, you will learn how to build an **API (Application Programming Interface)** using Node.js and Express.



Note: You will not connect this Back-End to your React app today. You will build the engine in isolation and test it using a professional developer tool: **Postman**.

Theory: What is CRUD?

When an application interacts with data, it generally performs four basic operations. In the industry, we call this **CRUD**:

- ✓ **Create**: Adding new data.
- ✓ **Read**: Retrieving existing data.
- ✓ **Update**: Modifying existing data.
- ✓ **Delete**: Removing data.

To tell the server which action we want to perform, we use **HTTP Methods** (or HTTP Verbs):

- GET is used to **Read**.
- POST is used to **Create**.
- PATCH or PUT are used to **Update**.
- DELETE is used to **Delete**.

To practice this, we are not going to touch the newspaper. Instead, we will build a completely different API in our sandbox: a **Tech Inventory Manager** to keep track of computers in a school.

Step 1: Initialize the Server

Open your terminal and create a new temporary folder for your sandbox:

```
mkdir backend-sandbox # Create a new directory for our backend sandbox
cd backend-sandbox # Change into the new directory
npm init -y # Initialize a new Node.js project with default settings
npm install express cors # Install the required packages
```

- ✓ `express`: A framework that makes creating servers in Node.js very easy.
- ✓ `cors`: A security package that will allow our future React app to talk to this server.

Step 2: The Database

Since we are not setting up a complex database like PostgreSQL today, we will use a simple `.json` file to permanently store our data.

Create a file named `inventory.json` in your folder and add some starting data:

```
[
  {
    "id": 1,
    "device": "MacBook Pro",
    "stock": 5
  },
  {
    "id": 2,
    "device": "Dell XPS 15",
    "stock": 12
  }
]
```

Step 3: The Boilerplate (Server Setup)

Create a file named `server.js`. This is the brain of your API.

We need to import Express, configure it to read JSON, and start listening for requests.

```
const express = require('express');
const fs = require('fs'); // Node's built-in File System module
const cors = require('cors');

const app = express();
app.use(cors());
app.use(express.json()); // Allows the server to understand JSON bodies

const PORT = 3000;

// We will add our routes (GET, POST, etc.) here!

app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```

To start your server, run `node server.js` in your terminal.



Important: Every time you modify `server.js`, you must stop the server (`Ctrl + C` or `Cmd + C` in the terminal) and restart it with `node server.js` to see the changes!

Step 4: The READ Endpoint (GET)

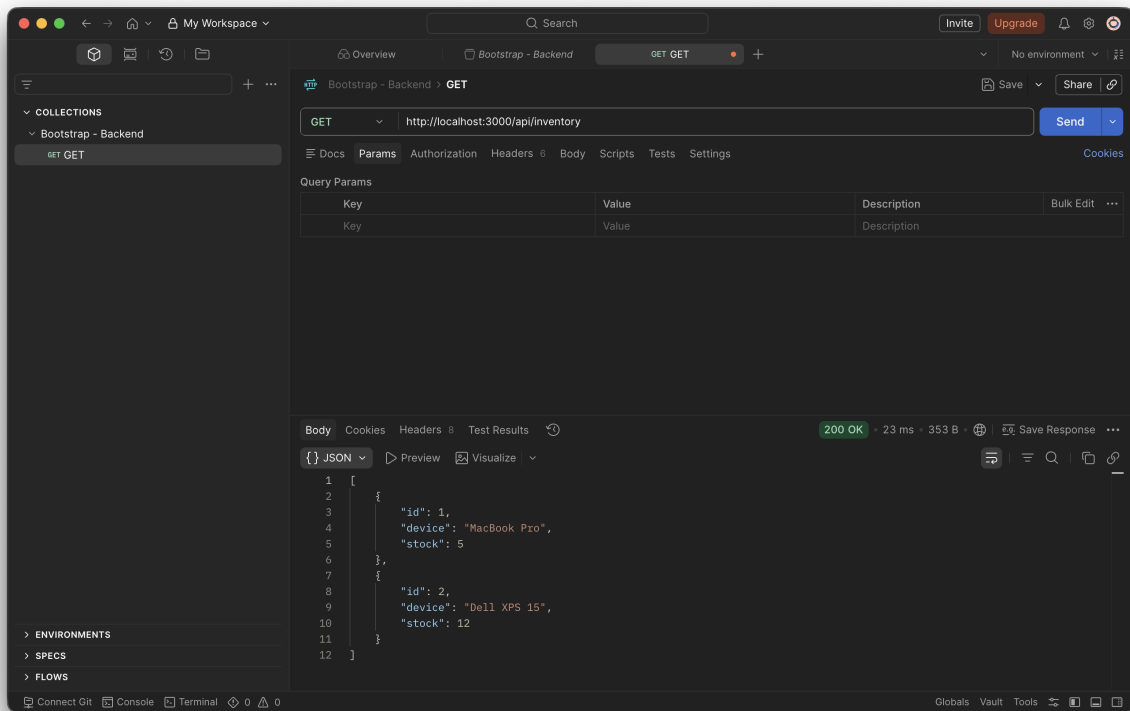
Let's allow users to see the inventory. Add this code inside `server.js` (before the `app.listen` block):

```
// GET: Retrieve all items
app.get('/api/inventory', (req, res) => {
  // 1. Read the file
  const rawData = fs.readFileSync('inventory.json');
  // 2. Convert it from text to a JavaScript array
  const inventory = JSON.parse(rawData);
  // 3. Send it back to the client
  res.status(200).json(inventory);
});
```

Testing with Postman

You cannot easily test APIs in a web browser. Developers use a tool called **Postman**.

1. Open Postman.
2. Select **GET** and enter `http://localhost:3000/api/inventory`.
3. Click **Send**. You should see your JSON array appear!



Step 5: The CREATE Endpoint (POST)

Now, let's allow the IT department to add new computers to the database.

```
// POST: Add a new item
app.post('/api/inventory', (req, res) => {
  // 1. Get the current inventory
  const inventory = JSON.parse(fs.readFileSync('inventory.json'));

  // 2. Create the new item using the data sent by the client (req.body)
  const newItem = {
    id: Date.now(), // Generates a unique ID based on the current timestamp
    device: req.body.device,
    stock: req.body.stock
  };

  // 3. Push it to the array
  inventory.push(newItem);

  // 4. Save the updated array back to the JSON file
  fs.writeFileSync('inventory.json', JSON.stringify(inventory, null, 2));

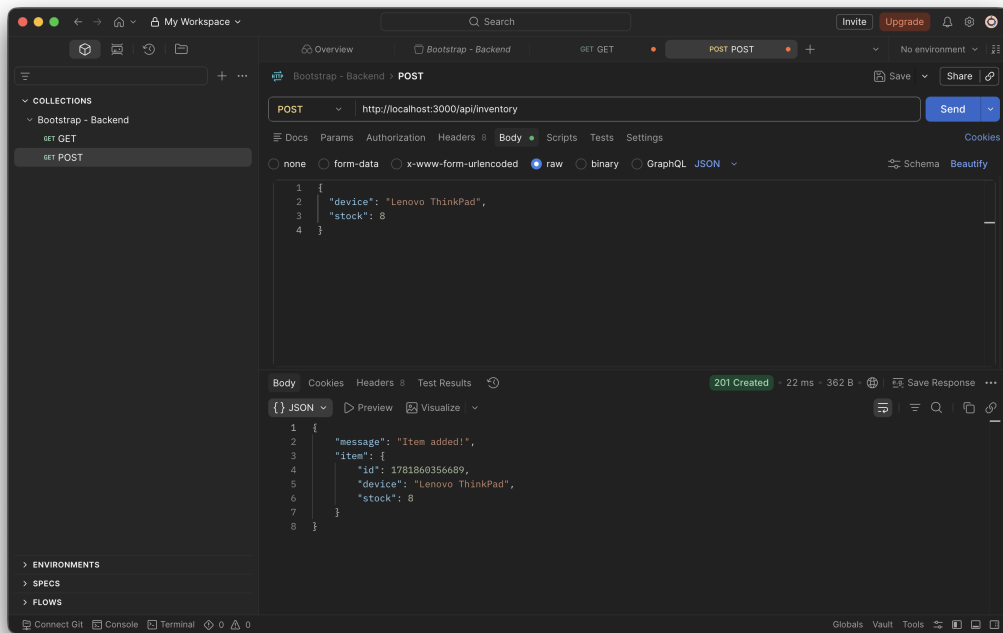
  // 5. Respond to the client
  res.status(201).json({ message: "Item added!", item: newItem });
});
```

Testing with Postman

1. Select **POST** and enter `http://localhost:3000/api/inventory`.
2. Go to the **Body** tab, select **raw**, and choose **JSON** from the dropdown.
3. Enter this payload:

```
{
  "device": "Lenovo ThinkPad",
  "stock": 8
}
```

4. Click **Send**. Look at your `inventory.json` file in VS Code. It was permanently updated!



After this, you can test the `GET` endpoint again to see the new item in the list. Also, as you can see, the `id` is automatically generated by the server. You don't have to worry about it when adding new items.

Step 6: The UPDATE & DELETE Endpoints (PATCH / DELETE)

To complete our CRUD, we need to be able to modify and delete data using the item's `id` in the URL (e.g., `/api/inventory/1`).

```
// PATCH: Update an item's stock
app.patch('/api/inventory/:id', (req, res) => {
  const inventory = JSON.parse(fs.readFileSync('inventory.json'));
  // Extract the ID from the URL (convert it to a number)
  const idToFind = parseInt(req.params.id);

  // Find the exact item in the array
  const item = inventory.find(i => i.id === idToFind);

  if (!item) {
    return res.status(404).json({ error: "Item not found" });
  }

  // Update the stock
  item.stock = req.body.stock;

  // Save the file
  fs.writeFileSync('inventory.json', JSON.stringify(inventory, null, 2));
  res.status(200).json({ message: "Stock updated!", item: item });
});

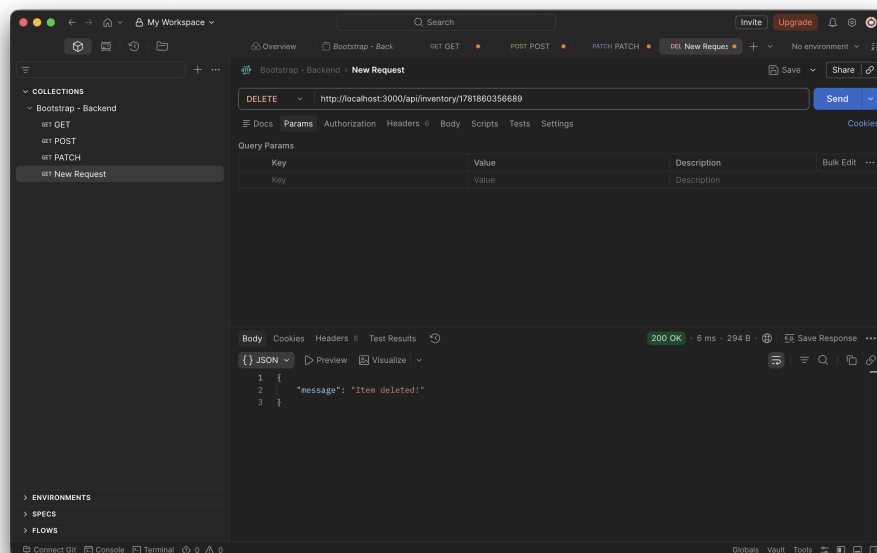
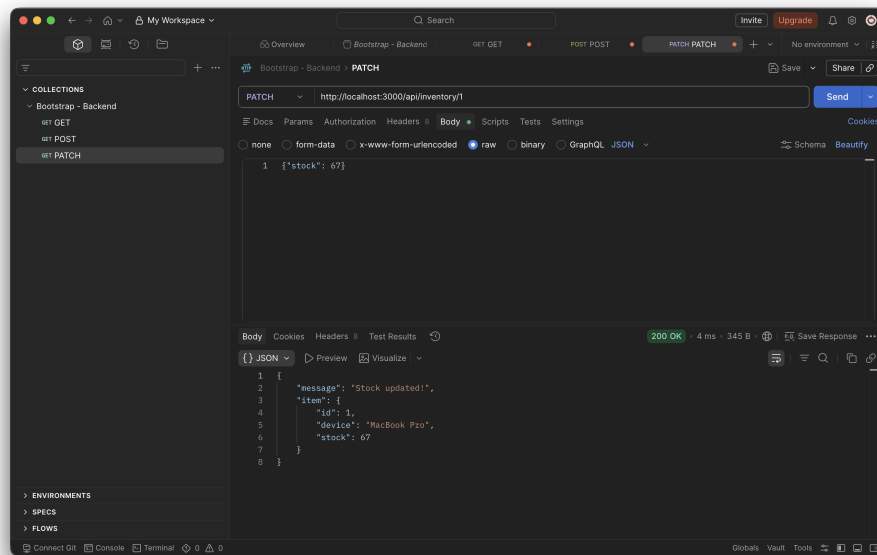
// DELETE: Remove an item
app.delete('/api/inventory/:id', (req, res) => {
  const inventory = JSON.parse(fs.readFileSync('inventory.json'));
  const idToDelete = parseInt(req.params.id);

  // Keep all items EXCEPT the one with the matching ID
  const newInventory = inventory.filter(i => i.id !== idToDelete);

  fs.writeFileSync('inventory.json', JSON.stringify(newInventory, null, 2));
  res.status(200).json({ message: "Item deleted!" });
});
```



Test it in Postman: Try making a PATCH request to `http://localhost:3000/api/inventory/1` with `{"stock": 50}` in the body. Then, try a DELETE request to the same URL!



Conclusion

Congratulations! You have just built a complete CRUD API. You know how to setup a server, read a JSON database, create new entries, modify them, and delete them.

Close your sandbox, open your main `newspaper-backend` project, and use these exact same concepts to build the Article API for the Editor-in-Chief!

v1

{EPITECH}